

1 Expresie - Soluție și Editorial

1.1 Enunțul problemei

Se dau n cifre (cifrele 0-9) și trei numere întregi p, q . Trebuie să selectăm o subsecvență contiguă de lungime $lg = n - p - q$ și să-i punem semnul $+$. Cifrele rămase sunt sortate crescător; primele q cifre vor avea semnul $-$, iar ultimele p cifre vor avea semnul $+$. Să se calculeze valoarea expresiei rezultate, ținând cont că cifrele pot forma numere mari (până la 1000 de cifre).

1.2 Observații cheie

Observația 1: Subsecvența care primește semnul $+$ trebuie să fie cea mai mare lexicografic posibil. Acest lucru este esențial deoarece, pentru a maximiza valoarea finală a expresiei, vrem ca partea adunată să fie cât mai mare.

Observația 2: Cifrele care nu fac parte din subsecvență sunt sortate. Prin urmare, cele mai mici q cifre vor fi scăzute, iar cele mai mari p cifre vor fi adăugate. Această strategie minimizează suma totală care se scade și maximizează suma care se adună.

Observația 3: Dacă $p > 0$, cifra de la poziția 0 (prima cifră din șir) este întotdeauna adăugată separat, deoarece face parte din grupul cifrelor cu semnul $+$, dar nu este inclusă în sortare (intră în calculul sumei inițiale).

Observația 4: Numerele pot avea până la 1000 de cifre, deci trebuie implementată aritmetica pe numere mari: adunare, scădere și comparare.

1.3 Algoritmul de rezolvare

Pasul 1: *Găsirea subsecvenței maxime lexicografice.*

- Parcurgem toate pozițiile posibile i de la 0 la $n - lg$.
- Comparăm subsecvența care începe la poziția i cu cea care începe la poziția curentă maximă $imax$, folosind comparație lexicografică.
- Păstrăm poziția $imax$ unde subsecvența este maximală.

Pasul 2: *Separarea cifrelor.*

- Cifrele din afara subsecvenței $[imax, imax + lg - 1]$ sunt copiate într-un vector auxiliar $a[]$.
- Dacă $imax = 0$, toate cifrele de la poziția lg până la $n - 1$ formează restul; suma inițială este 0.
- Altfel, cifra $s[0]$ este adăugată la sumă, iar cifrele $s[1]$ până $s[imax - 1]$ sunt incluse în rest.

Pasul 3: *Sortarea și atribuirea semnelor.*

- Sortăm vectorul $a[]$ crescător.

- Primele q cifre ($a[0..q-1]$) primesc semnul $-$.
- Următoarele p cifre ($a[q..nr-1]$) primesc semnul $+$.

Pasul 4: *Calculul expresiei.*

- Fie S suma calculată din cifrele cu semne (pozitive și negative).
- Convertim subsecvența maximală într-un număr mare (cifrele în ordine inversă).
- Dacă $S \geq 0$, adunăm S la subsecvență.
- Dacă $S < 0$, comparăm $|S|$ cu subsecvența: dacă $|S| \leq$ subsecvența, scădem; altfel, scădem din $|S|$ și punem semnul $-$.

Pasul 5: *Afișarea rezultatului* ca număr mare (cifre în ordine inversă afișării).

1.4 Note de implementare

Structura NrMare: Reprezentarea unui număr mare este un vector de int-uri, unde fiecare element reprezintă o cifră (unități, zeci, etc.), cu cifrele memorate în ordine inversă (LSB primul).

Funcția adun: Realizează adunarea a două numere mari. Completați cel mai scurt număr cu zerouri la stânga, apoi adunați cifre cu cifre, propagând transportul.

Funcția dif: Realizează scăderea $x - y$, presupunând $x \geq y$. Completați cu zerouri, apoi scădeți cifre cu cifre, împrumutând când este necesar. La final, eliminăm zerourile nesemnificative de la sfârșit.

Funcția comparnr: Compară două numere mari. Mai întâi comparăm lungimile; dacă sunt egale, parcurgem cifrele până la prima diferență.

Funcția compars: Compară două subsecvențe lexicografic. Parcurgem simultan caracterele din cele două subsecvențe până găsim o poziție unde diferă.

1.5 Corectitudinea soluției

Lema 1: Subsecvența maximală lexicografică dă cea mai mare valoare posibilă pentru partea adunată a expresiei.

Lema 2: Sortând cifrele rămase și atribuind semnul $-$ celor mai mici q și semnul $+$ celor mai mari p , obținem valoarea maximă a sumei provenite din cifrele neselectate.

Lema 3: Calculul final folosind aritmetica pe numere mari corectează valoarea expresiei, ținând cont de semnul sumei $|S|$.

Din aceste leme rezultă că algoritmul produce valoarea corectă a expresiei.

1.6 Complexitate

- Găsirea subsecvenței maxime: $O(n \cdot lg) = O(n^2)$ în cel mai rău caz, dar $lg \leq 1000$ și $n \leq 1000$, deci acceptabil.
- Sortarea cifrelor rămase: $O(n \log n)$.
- Operațiile pe numere mari: $O(lg)$ pentru fiecare operație.

Complexitatea totală este $O(n^2 + n \log n) \leq O(10^6)$, cu memorie $O(n)$.

1.7 Codul sursă

```
//prof. Em. Cerchez
#include <fstream>
#include <algorithm>
#define LGMAX 1002

using namespace std;
ifstream fin("expresie.in");
ofstream fout("expresie.out");
int n, p, q, lg, suma, lgsum, lgrez;
char s[LGMAX];
char a[LGMAX];

typedef int NrMare[LGMAX];
NrMare secv, sum, rez;

void adun(NrMare x, int lgx, NrMare y, int lgy, NrMare z,
int& lgz);
void dif(NrMare x, int lgx, NrMare y, int lgy, NrMare z, int
& lgz);
void afisare(NrMare x, int lgx);
int compars(int i, int j);
int comparnr(NrMare x, int lgx, NrMare y, int lgy);

int main()
{int i, j, nr, imax, k, semn;
fin>>n>>p>>q>>s;
lg=n-p-q;
//determin secventa de maxima lexicografica
imax=0;
if (p>0)
{for (i=1; i<n-lg+1; i++)
if (compars(i,imax)>0) imax=i;
}
//cifrele de secventa maxima sunt pe pozitile imax..imax+
lg-1
nr=0;
if (imax==0) //secventa este la inceput
```

```

    {for (j=imax+lg; j<n; j++) a[nr++]=s[j]; suma=0;}
    else
    {
        p--; //plus se pune inaintea acestei secvente
        suma=s[0]-'0';
        for (j=1; j<imax; j++) a[nr++]=s[j];
        for (j=imax+lg; j<n; j++) a[nr++]=s[j];
    }

//sortez celelalte cifre
sort(a,a+nr);
//primele q vor fi cu minus, ultimele p cu plus
for (i=0; i<q; i++) suma=suma-(a[i]-'0');
for (i=q; i<nr; i++) suma=suma+(a[i]-'0');
if (suma<0) {suma=-suma; semn=-1;}
    else semn=1;
//adunam sum cu secventa maxima
for (k=0, j=imax+lg-1; j>=imax; j--, k++) secv[k]=s[j]-'0';
do
    {sum[lgsum++]=suma%10; suma/=10; }
while (suma);
if (semn==1)
    adun(secv, lg, sum, lgsum, rez, lgrez);
    else
        if (comparnr(secv,lg,sum,lgsum)>=0) {dif(secv,lg,sum,
            lgsum, rez, lgrez); semn=1;}
            else
                dif(sum,lgsum,secv,lg,rez,lgrez);
if (semn==-1) fout<<'-' ;
afisare(rez, lgrez);
return 0;
}

```